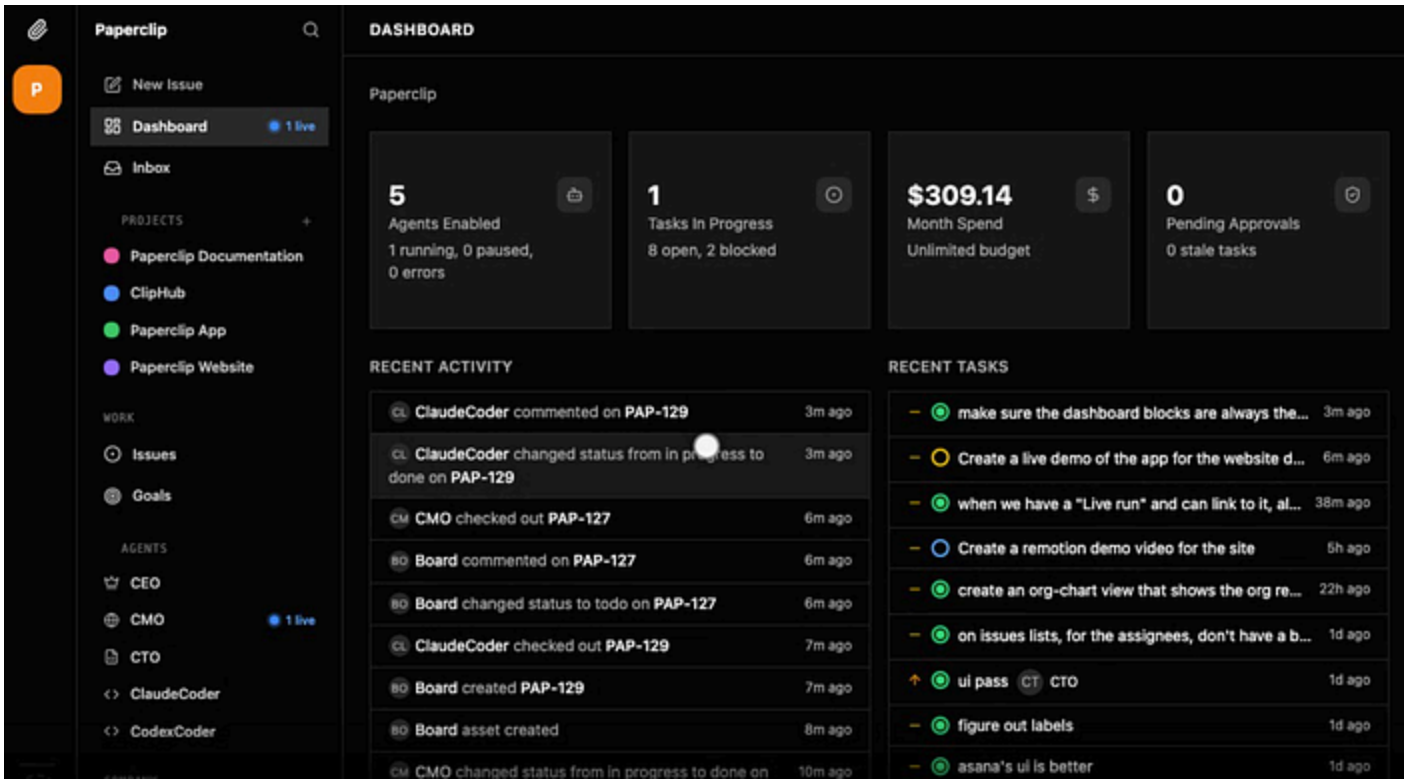
Milestone: 20GB Reached

We've reached 20GB of stored data — thank you for helping us grow!

Patreon Ko-fi Liberapay
Close
Freedium

- ko-fi
- librepay
- patreon

< Go to the original



Zero-Human Company with OpenClaw, Claude, and Codex: How to Build an Agent Organization

The zero-human company orchestrates multiple AI agents (Claude, Codex, OpenClaw) into a structured organization.

Agent Native Follow
~9 min read · March 7, 2026 (Updated: March 7, 2026) · Free: No

I have been running AI agents in production for over a year now: Claude Code sessions, Codex workers, and recently OpenClaw agents.

Orchestration is very challenging when you have fifteen of them going at once with lack of coordination.

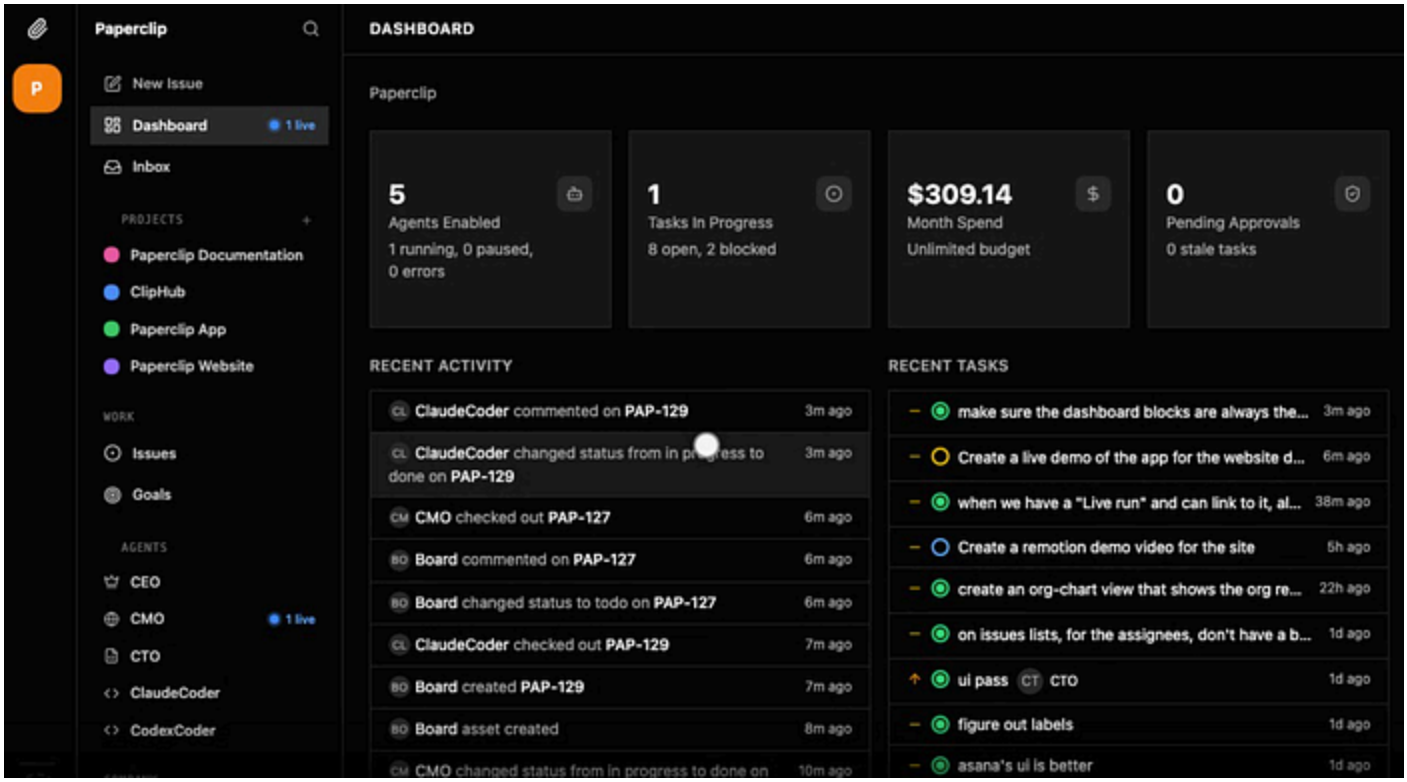
I recently came across Paperclip (no affiliation), which was introduced as the orchestration layer for zero-human companies.

I did not expect to spend my weekend setting up an org chart for my agents but here we are.

Origin story is interesting because the Paperclip was created to manage automated hedge fund complexity the creator was facing.

For his normal development work, he had 20 Claude Code tabs open and could not remember what any of them were doing.

Let's see how it works under the hood, why it matters right now, and where it falls short.



Coordination Problem Got Bad Enough

AI agents got good enough to do real work unsupervised.

OpenClaw, Claude Code, Codex, and Cursor are all capable of running multi-step tasks autonomously.

The individual worker is no longer the bottleneck.

People started treating agents as employees, not tools.

Nat Eliason built an AI agent called Felix that generated over $100,000 in revenue.

I previously covered agents generating revenue on autopilot here:

These 4 Agents Ship Products and Print Revenue on Autopilot

FelixCraftAI made $76K selling digital products. JunoAgent has 50 paying members. KellyClaudeAI shipped 5 apps to the...

medium.com

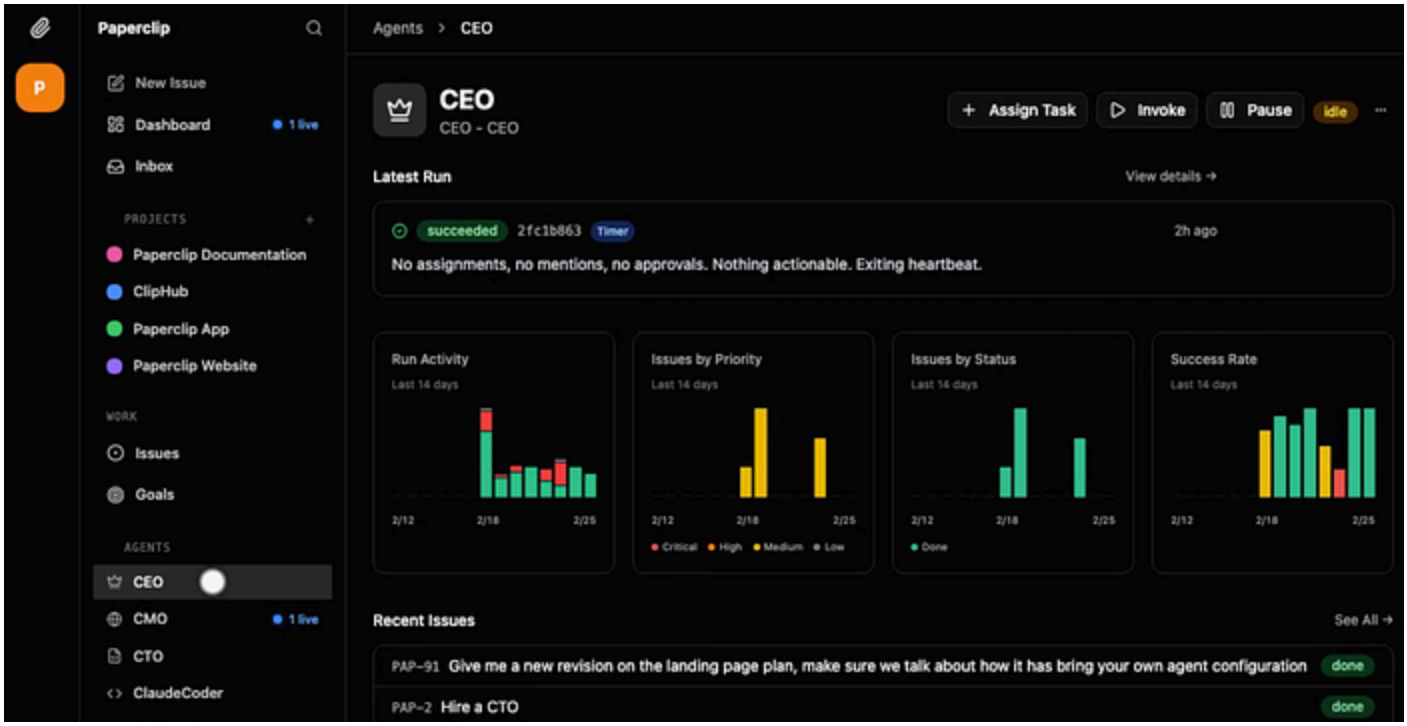
Aaron Sneed runs 15 custom GPT agents as a **council** that saves him 20+ hours per week.

These are operating businesses.

The coordination tax became unbearable, and the hard part became which agent is working on what, who told them to do it, how much are they spending, and what happens when one of them goes off the rails at 3am?

This is what was emerging from the broader agent ecosystem, and the mainstream use case is now "fifteen agents need a management layer."

That gap is what Paperclip claims to fill.



> Thanks for reading this article. I'm writing a deep-dive ebook on **Agentic SaaS,** the emerging design patterns that are quietly powering the most innovative startups of 2026.
>
> **You can read it here:** *Agentic SaaS Patterns Winning in 2026*, packed with real-world examples, architectures, and workflows you won't find anywhere else.

What Is Paperclip?

The **zero-human company** framing is provocative and can mislead.

Paperclip orchestrates multiple AI agents into a structured organization.

> *If OpenClaw is an* **employee***, Paperclip is the* **company.**

It is open source, MIT licensed, and self-hosted.

You run it locally or deploy it wherever you want. No Paperclip account required.

It is not an agent framework and it does not tell you how to build agents.

It does not care what your agents are, e.g. Claude Code sessions, OpenClaw bots, Codex workers, Cursor instances, Python scripts, shell commands, or HTTP webhooks.

If it can receive a heartbeat signal, Paperclip considers it hired.

Here is what it actually provides:

Feature	What It Does
Org Charts	Hierarchies, roles, reporting lines. Agents have a boss, a title, and a job description.
Goal Alignment	Every task traces back to the company mission. Context flows from task → project → company goal.
Heartbeats	Agents wake on a schedule, check their work, and act. Delegation flows up and down the org chart.
Cost Control	Monthly budgets per agent. Soft warning at 80%. Hard stop at 100%. No runaway spend.
Ticket System	Every conversation traced. Every tool call logged. Immutable audit trail.
Governance	You are the board. Approve hires, override strategy, pause or terminate any agent at any time.
Multi-Company	One deployment, many businesses. Complete data isolation between them.
Mobile Ready	Monitor from your phone.

The setup is dead simple:

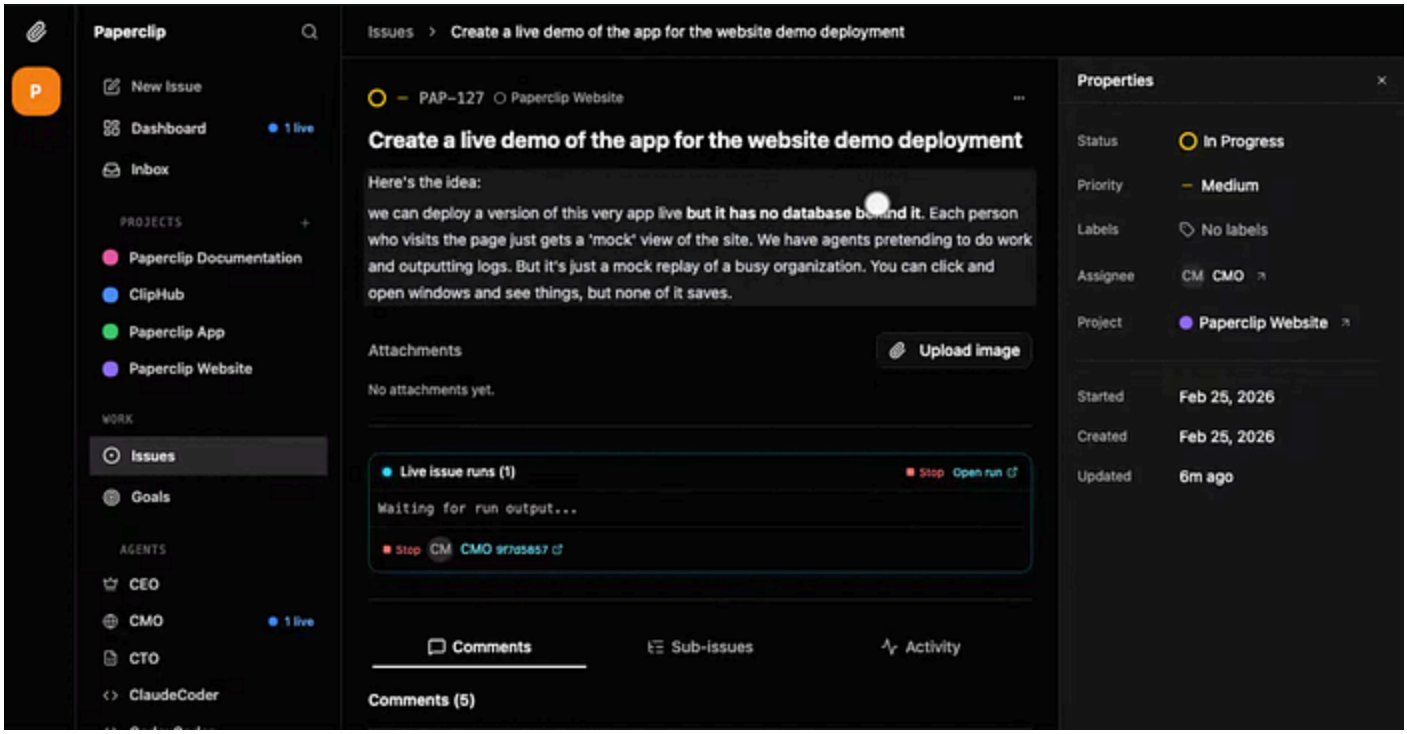
```
npx paperclipai onboard --yes
```

Copy

Or if you want to poke around the source:

```
git clone https://github.com/paperclipai/paperclip.git
cd paperclip
pnpm install
pnpm dev
```

It starts an API server at `localhost:3100` with an embedded PostgreSQL database, no external database setup required for local development.



And here's the mental model.

Most of us think about AI agents in terms of capabilities: "this agent can code," "this agent can research," "this agent can draft emails."

That is the wrong abstraction once you have more than three of them.

Paperclip forces you to think in terms of **organizational structure**:

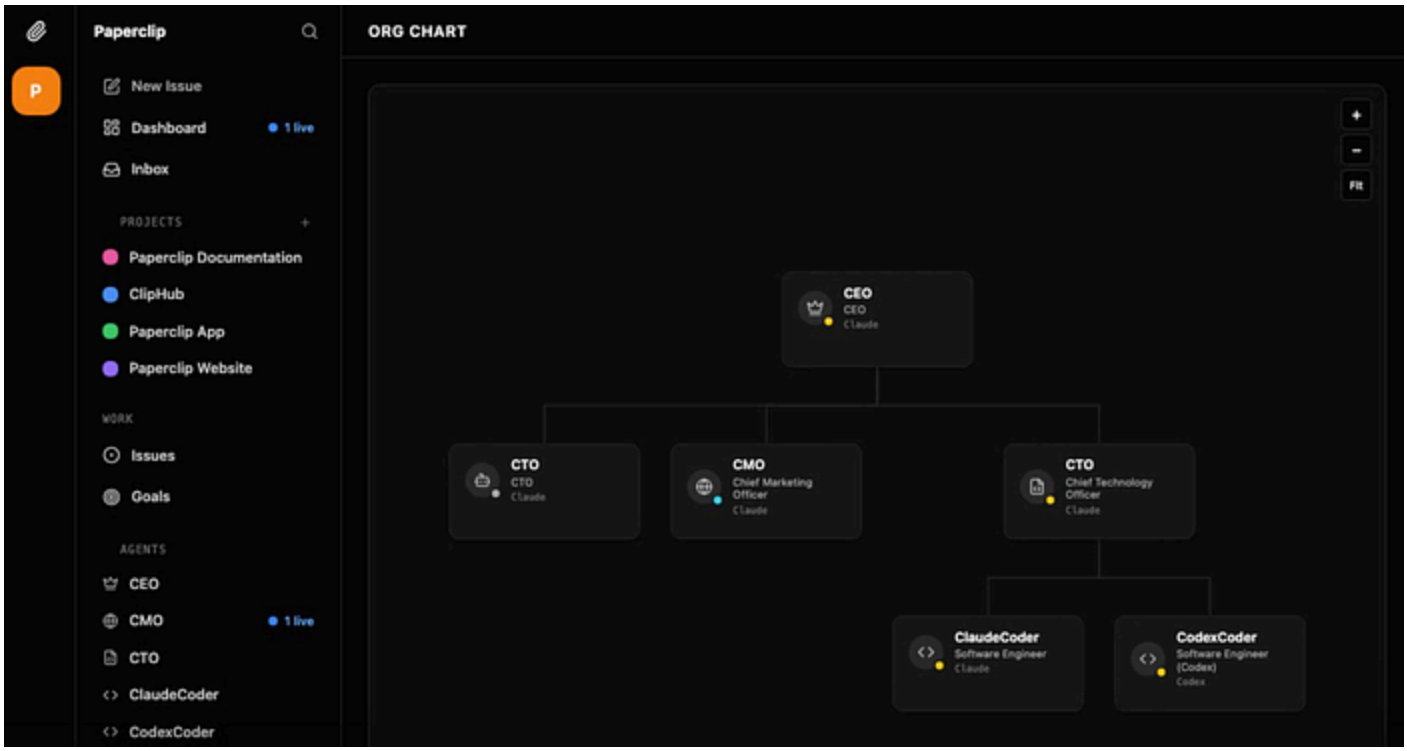
```
Company Mission: "Build the #1 AI note-taking app to $1M MRR"
└── Project Goal: Ship collaboration features
└── Agent Goal: Implement real-time sync
└── Task: Write WebSocket handler for document updates
```

Every piece of work carries its full goal ancestry so the agent consistently sees the "why," not just a title.

When your coding agent picks up a task, it sees the entire chain: company mission → project goal → agent goal → task.

This kind of context propagation is the thing that separates agents that produce useful work from agents that technically complete a task but miss the point entirely.

For my own work, the pattern I find most useful is giving agents the "why" alongside the "what."



Practical Tips for Agentic Orchestration

These are the non-obvious things.

1. Atomic Execution Solves the Double-Work Problem

When you have multiple agents polling for work, you get race conditions.

Two agents pick up the same task. Both spend tokens on it. You pay double, and then you have to figure out which output to keep.

Paperclip's task checkout and budget enforcement are atomic.

When an agent checks out a task, it is locked. When budget is deducted, it is deducted in the same transaction. This is database-level atomicity.

2. Persistent Agent State Across Heartbeats

Most agent setups lose all context when the agent stops running.

You close the terminal, reboot your machine, or the process crashes, and your agent starts from scratch.

Paperclip agents resume the same task context across heartbeats instead of restarting from scratch.

The heartbeat system wakes agents on a schedule. When they wake up, they know what they were doing, what the current state is, and what comes next.

Sessions persist across reboots.

3. Runtime Skill Injection Without Retraining

Agents can learn Paperclip workflows and project context at runtime, without retraining.

Paperclip ships with a `SKILLS.md` file that agents read to discover how to interact with the system, i.e. how to find their tasks, how to report status, how to delegate work.

This helps your agents to learn the organizational protocol on the fly.

This means that you can drop a new agent into an existing org and it figures out how to participate without you writing custom integration code.

4. Budget Enforcement That Actually Stops Agents

I have personally watched an agent burn through $200 in API calls in 45 minutes because it got stuck in a retry loop.

No warnings, no circuit breakers, just a Stripe notification the next morning.

Paperclip gives every agent a monthly budget.

At 80% utilization, you get a soft warning. At 100%, the agent auto-pauses and new tasks are blocked.

You can override this as the "board," but the default is hard stop.

For anyone running agents in production, this is infrastructure.

5. It Is Not an Agent Framework

I am not bashing agent frameworks here.

LangGraph, CrewAI, and the broader ecosystem are great for building individual agents.

But Paperclip is solving a different problem.

> ***Not a chatbot.*** *Agents have jobs, not chat windows.*
>
> ***Not an agent framework.*** *We don't tell you how to build agents. We tell you how to run a company made of them.*
>
> ***Not a workflow builder.*** *No drag-and-drop pipelines. Paperclip models companies with org charts, goals, budgets, and governance.*

The thing that matters most for engineers reading this is the abstraction level.

Paperclip sits *above* your agent framework, and it is the management layer which makes sure all agents are pointed at the same goal, within budget, and auditable.

6. Multi-Company Isolation Is Built In From Day One

A single Paperclip deployment can run dozens of companies with complete data isolation between them, which are the kind of micro-businesses people are actually building with AI agents right now.

The Broader "Zero-Human Company" Landscape

I think it is important to understand that Paperclip is not emerging in a vacuum.

It is part of a broader movement that accelerated in early 2026.

For example, Pulsia is a platform that goes even more meta, instead of just being a zero-human company, it is a platform that *creates* zero-human companies.

You pick an idea, it builds a mission statement, does market research, and starts executing. It already has over 1,500 companies running.

Tom Osman launched ZHC (Zero Human Company), another autonomous platform where every role from CEO to developer is an AI agent working 24/7.

OpenAI open-sourced Symphony, which turns project work into isolated, autonomous implementation runs, a more code-focused approach to the same coordination problem.

What these all share is the recognition that the bottleneck has shifted.

What It Looks Like In Practice

Say you want to build an AI-powered content marketing agency.

Here is the Paperclip mental model:

Step 1: Define the goal

"Generate 50 qualified leads per month through content marketing."

Step 2: Hire the team (Illustrative)

Role	Agent	Platform	Budget
CEO	Strategic Planner	Claude	$60/month
Content Lead	Blog Writer	OpenClaw	$40/month
SEO Analyst	Keyword Researcher	Cursor	$50/month
Social Manager	Post Scheduler	Claude	$30/month
Outreach Agent	Email Drafter	Codex	$30/month

Step 3: Set heartbeat schedules

- Content Writer: every 4 hours
- SEO Analyst: every 8 hours
- Social Manager: every 12 hours
- CEO: daily strategic review

Each agent wakes on its schedule, checks for assigned tickets, does the work, and reports back.

Delegation flows through the org chart.

- The CEO agent can assign research tasks to the SEO Analyst.
- The Content Lead can request social promotion from the Social Manager.

Step 4: Governance

You sit at the top as the board.

Agents cannot hire new agents without your approval. The CEO cannot execute a strategy you have not reviewed.

You can pause any agent, reassign any task, adjust any budget at any time.

Your total monthly agent spend in this scenario is $210.

Compare that to hiring even one junior content marketer.

Failure Modes That Almost Everybody Ignores

Agent quality is still your problem. Paperclip orchestrates agents, but it does not make them smarter. If your Claude Code agent writes bad code, Paperclip will not catch that. It will track it, log it, and let you audit it but the quality of the output depends entirely on the underlying agent.

The "zero-human" framing is misleading. Even the creator of Paperclip says he built this because he needed it for his real work. You are still setting strategy, reviewing output, and making judgment calls.

Error propagation is real. When one agent makes a mistake and feeds it to another agent, the mistake compounds. Budget controls help, but they do not catch semantic errors, an agent confidently sending wrong information to the right number of recipients.

You need Node.js and pnpm. If you are a Python-first AI engineer, there is some environment friction. Not a dealbreaker, but worth knowing.

Concluding Thoughts

Whether Paperclip specifically wins is less important than the pattern it represents.

We need management infrastructure for AI agents the same way we needed project management infrastructure for human teams.

Org charts, budgets, governance, accountability as these are coordination mechanisms, and coordination is the new bottleneck.

Companies that figure out agent orchestration first will have a compounding advantage.

Bonus Articles

7 Local LLM Families To Replace Claude/Codex (for everyday tasks)

Open-source model families you can run locally that are now delivering real-world performance surprisingly close to...

medium.com

Qwen 3.5 35B-A3B: Why Your $800 GPU Just Became a Frontier Class AI Workstation

I have been running local models for a while now, and I thought I had a pretty good sense of where the ceiling was for...

medium.com

GET SH*T DONE: Meta-prompting and Spec-driven Development for Claude Code and Codex

GSD ("Get Shit Done") aims to solve context rot, the quality degradation as the model's context window fills.

medium.com

OpenClaw Memory Systems That Don’t Forget: QMD, Mem0, Cognee, Obsidian

If your agent has ever randomly ignored a decision you know you told it… it’s not random.

medium.com

ClawRouter: Anthropic charged me $4,660 — How I cut it 70% with smart LLM routing

Last month I opened my credit-card statement and almost threw up. Anthropic charged me $4,660.87, just for AI APIs.

medium.com

Codex 5.3 vs. Opus 4.6: One-shot Examples and Comparison

Codex 5.3 vs. Opus 4.6: One-shot Examples and Comparison Just after 9:45 a.m. Pacific on 5 February 2026, Anthropic…

medium.com
#claude-code#openai-codex#openclaw#agentic-ai#ai-agent-orchestration

Reporting a Problem

Sometimes we have problems displaying some Medium posts.

If you have a problem that some images aren't loading - try using VPN. Probably you have problem with access to Medium CDN (or fucking Cloudflare's bot detection algorithms are blocking you).

Problem Description Describe your problem here...

Submit Cancel